Smart Delivery Route Optimization

Assignment 5 | Strategy Pattern | Java 17 | Implementation report

The engine selects a delivery route from eight supplied alternatives. Five interchangeable strategies optimize time, cost, environmental impact, a weighted balance, or distance. Every strategy first applies the request constraints and then explains its selection. The implementation passes 51 automated JUnit 5 tests.

Architecture

RoutePlanner is the Context and depends only on **RouteStrategy**. Its plan method delegates to the selected object; setStrategy changes that object while the program is running. FastestRouteStrategy, CheapestRouteStrategy, EcoFriendlyRouteStrategy, BalancedRouteStrategy, and DistanceFirstStrategy implement the algorithms through the shared AbstractRouteStrategy base class.

StrategySelector selects an algorithm from an injected priority mapping. ConstraintChecker evaluates feasibility. RouteDecision carries the winner, metrics, score, explanation, and assessments of every candidate. Returning this record instead of a bare Route preserves the explanation required by the assignment.

Domain and demonstration data

Location, Route, and DeliveryRequest are validated immutable records. Route includes cost components, traffic and weather delays, availability, and prohibited priorities. The request supplies endpoints, budget, deadline, emission limit, eco preference, and blocked route IDs. Currency is KZT; emissions are estimated kg CO2. Demonstration values are fictional.

Route	Minutes	KZT	Km	CO2 kg	Availability
R01	40	3850	12	4.50	Available
R02	25	6220	17	6.50	Available
R03	50	1950	20	3.80	Available
R04	38	3050	15	0.80	Available
R05	58	850	10	0.10	Not for URGENT
R06	32	2975	13	2.00	Available
R07	47	3210	8	3.00	Available
R08	10	300	5	0.05	Unavailable

The table uses STANDARD request 105: budget 10000 KZT, deadline 120 minutes, maximum emissions 20 kg. Times include traffic and weather; prices include the STANDARD surcharge. R08 is rejected before scoring.

Algorithms and feasibility

Fastest and cheapest

```
time = baseMinutes + trafficDelayMinutes + weatherDelayMinutes
cost = baseCost + distanceKm * costPerKm + prioritySurcharge + fees
```

Fastest minimizes effective time, rather than raw travel time. Cheapest minimizes the full request-specific cost. The priority surcharge is 500 KZT for URGENT, 150 for STANDARD, and zero for ECONOMY or ECO. Monetary components use BigDecimal; the final sum is rounded once to two decimal places with HALF_UP.

Environmental scoring

```
ecoScore = emissionKg * emissionWeight + distanceKm * distanceWeight
default emissionWeight = 1.0; default distanceWeight = 0.05
```

EcoFriendlyRouteStrategy owns the environmental policy and its configurable weights. The Context does not need to know the units or coefficients. Weights must be finite and non-negative, with at least one positive. The score is a policy index, not a physical estimate of additional CO2.

Balanced scoring

```
normalized(x) = (x - min) / (max - min)
normalized(x) = 0 when max == min
score = nt*0.40 + nc*0.30 + nd*0.10 + ne*0.20
```

All four ranges are computed from feasible routes only. Normalization makes the units comparable. The constructor accepts configurable time, cost, distance, and emission weights; each must be finite and between zero and one, and their sum must equal one within 1e-9. Zero weights are valid. Equal values give zero contribution, avoiding division by zero.

For R06, normalized time, cost, distance, and emissions are 0.2121, 0.3957, 0.4167, and 0.2969. Its score is 0.3046, the lowest in request 105. Changing the weights to (1, 0, 0, 0) chooses the fastest route R02.

Hard constraints and explicit failures

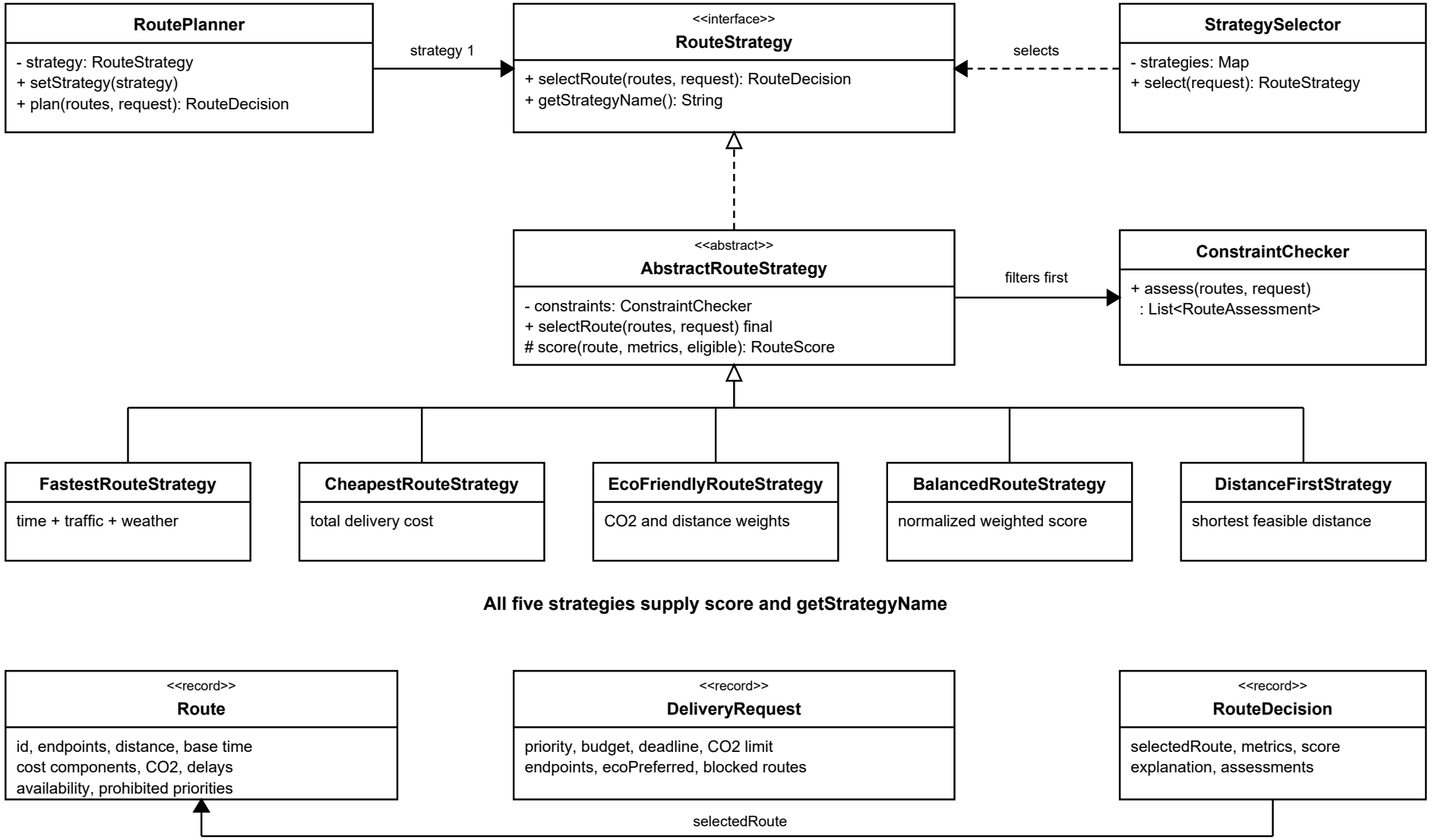
ConstraintChecker collects every rejection reason: unavailability, endpoint mismatch, cost above budget, effective time above deadline, emissions above the limit, request-specific blocking, or prohibited priority. Equality with a limit is permitted. Rejected routes have no score, so an attractive but forbidden candidate cannot win.

An empty list or an entirely infeasible set throws NoSuitableRouteException with candidate assessments. Invalid weights and numeric data throw IllegalArgumentException; required null data fail explicitly. Duplicate route IDs and time overflow are also rejected. Every successful strategy uses ascending route ID to resolve exact score ties.

Fifth strategy

DistanceFirstStrategy minimizes distance after the same feasibility checks. It was added in separate commit 9baf7b3. Only the existing DemoData composition class needed modification to register the new algorithm; RoutePlanner and StrategySelector stayed unchanged.

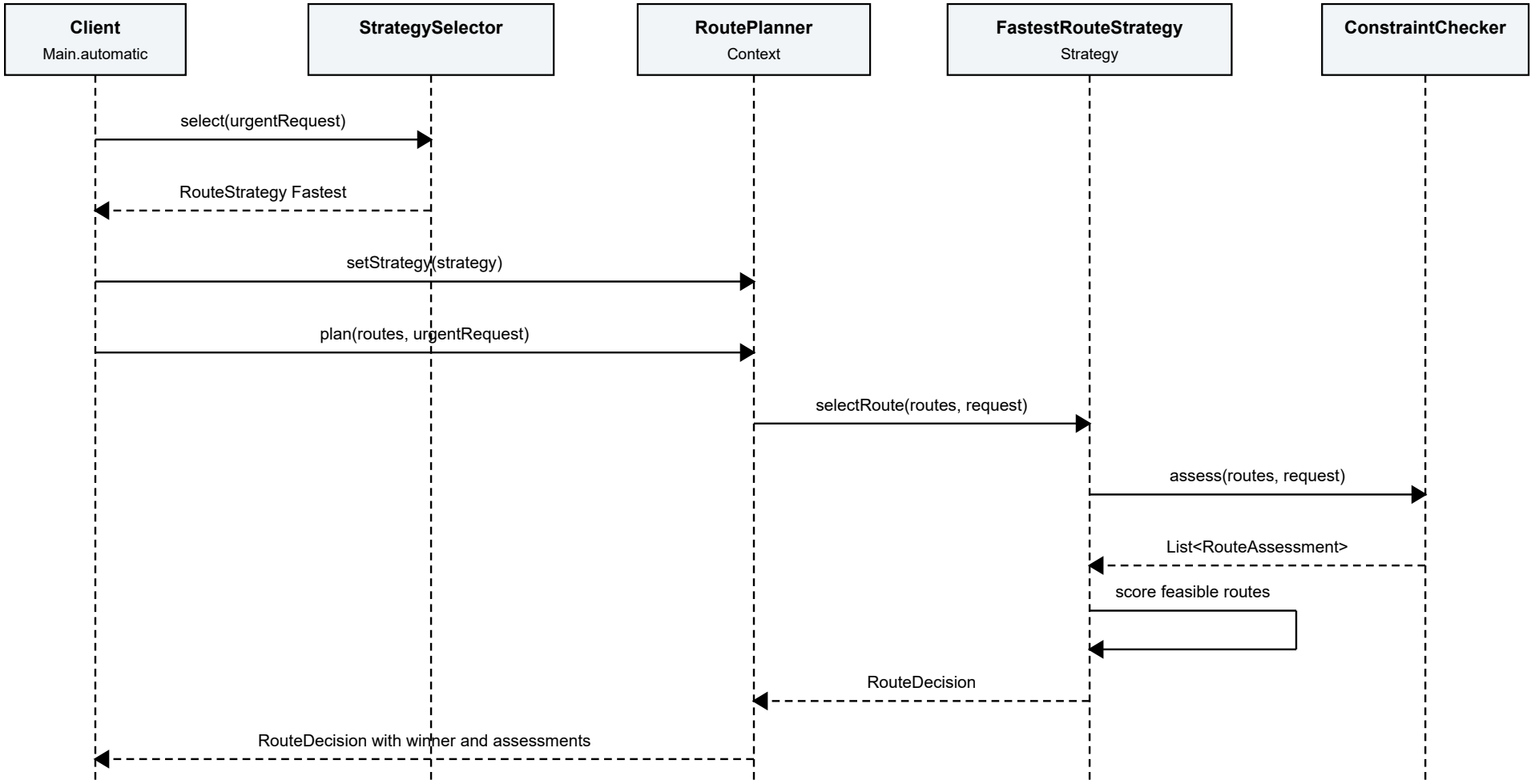
UML class diagram



Route and DeliveryRequest are selectRoute parameters; RouteDecision is its return type.

Solid hollow triangles represent inheritance; the dashed hollow triangle represents interface realization. Solid arrows show references or delegation. The Context holds the Strategy abstraction. The Mermaid source includes additional metrics, assessment, and weight relationships.

UML sequence diagram



selectRoute executes the shared pipeline inherited from AbstractRouteStrategy.

This sequence corresponds to Main.automatic for an urgent request. StrategySelector returns a FastestRouteStrategy instance. The client installs it in RoutePlanner and calls plan. The inherited strategy pipeline checks constraints, scores feasible routes, and returns a RouteDecision.

Demonstration and test evidence

Comparison mode

Main.compare executes all five strategies on the same request and candidate set. Selected fields from the console output are shown below. Scores use different units and are compared only within their own strategy.

```
Fastest        -> R02 | 25 min | 6220.00 KZT | score 25.0000
Cheapest       -> R05 | 58 min |  850.00 KZT | score 850.0000
Eco Friendly   -> R05 | 58 min |  850.00 KZT | score 0.6000
Balanced       -> R06 | 32 min | 2975.00 KZT | score 0.3046
Distance First -> R07 | 47 min | 3210.00 KZT | score 8.0000
```

Full output in docs/console-output.txt also lists all feasible scores and rejection reasons. For example, Fastest explains 20 base + 3 traffic + 2 weather = 25 minutes; all strategies reject R08 as unavailable.

Switching and automatic selection

Main.switching uses one RoutePlanner, initially Fastest, then performs three setStrategy calls to use Cheapest, Eco, and Balanced. The same Fastest algorithm changes from R02 to R03 when the budget drops from 10000 to 2500 KZT.

Main.automatic demonstrates URGENT to Fastest, ECONOMY to Cheapest, eco preference to Eco, and STANDARD to Balanced. Urgency takes precedence over eco preference. StrategySelector decides which algorithm to use; a strategy then chooses the route. Main.failures demonstrates empty input, no feasible route, invalid weights, and a negative budget.

Automated verification

Test class	Count	Coverage
StrategyTest	17	Four original algorithms, formulas, weights, normalization, ties and explanations
ConstraintsTest	17	All hard constraints, exact limits, invalid data, empty input, no valid route
ArchitectureTest	12	Runtime switching, interface delegation, selection rules and comparison
DistanceFirstStrategyTest	5	Fifth algorithm, filtering, substitution, ties and registration

```
Tests run: 51, Failures: 0, Errors: 0, Skipped: 0
Maven test/package: BUILD SUCCESS
```

Both build.ps1 and Maven were executed on OpenJDK 25.0.4 with Java 17 API and bytecode targeting. Class format is 61. Execution on an actual JDK 17 was not performed in this session. Test results copied from Maven are in docs/test-results.txt. The Linux/macOS build.sh is supplied; the executed standalone script was build.ps1.

Design rationale and use

Why Strategy fits

The application swaps the algorithm for the same route-selection operation. RoutePlanner never switches on algorithm names or inspects concrete strategy types. Business-policy conditions stay in StrategySelector and the Map supplied by the composition root. Adding a new ranking algorithm requires a new class, focused tests, and registration.

SOLID principles

SRP: planning, feasibility, ranking, policy selection, and console presentation have separate owners. **OCP:** DistanceFirstStrategy was introduced without changing planning or selection logic. **DIP:** the Context field, constructor, and setter depend on RouteStrategy. Concrete types are assembled in DemoData.

Strategy compared with Bridge

Strategy replaces the algorithm used to solve one problem. Bridge separates an abstraction hierarchy from an implementation hierarchy so both can vary independently, for example delivery services and notification channels. This engine varies route-ranking algorithms. The comparison concerns the patterns and does not assume a particular implementation in a previous assignment.

Limitations and next improvements

The engine ranks supplied routes for one request. It does not discover paths on a road graph, optimize a vehicle fleet, or update traffic in real time. Balanced currently recomputes ranges per candidate, giving O(n^2) work; precomputing four ranges once would reduce it to O(n). Its scores depend on the feasible set and can change when another feasible candidate is added. Other strategies run in O(n).

The mutable RoutePlanner is intended for one client thread; concurrent requests should use separate planners. Location matching uses exact record equality. The enum defines a closed set of delivery categories; adding a new category requires a new enum value and selector mapping, even though adding an algorithm for an existing category does not change the selector.

Build and submit

```
Windows: powershell -ExecutionPolicy Bypass -File .\build.ps1 -Run
Maven:   mvn package
Run:     java -jar target/smart-delivery.jar --compare
```

The download package includes an executable dist/smart-delivery.jar, complete sources, UML SVGs and Mermaid sources, test evidence, a Russian defense guide, and a Git history bundle. Publish the repository as public and submit its URL together with README or this PDF in Moodle. The local history contains meaningful commits made during this implementation, including the separate fifth-strategy commit; dates have not been backdated.

Technical references

JUnit 5.11.4 User Guide | Maven Compiler Plugin | Maven Surefire